# Planar Convex Hull Algorithms

1. The Convex Hull
   - problem statement
   - the Graham-Andrew algorithm in CGAL
2. an Incremental Algorithm
   - the lower and upper hull
   - formulation of the algorithm
   - correctness
   - cost analysis

MCS 481 Lecture 2
Computational Geometry
Jan Verschelde, 26 August 2026

# Planar Convex Hull Algorithms

1. The Convex Hull
   - problem statement
   - the Graham-Andrew algorithm in CGAL
2. an Incremental Algorithm
   - the lower and upper hull
   - formulation of the algorithm
   - correctness
   - cost analysis

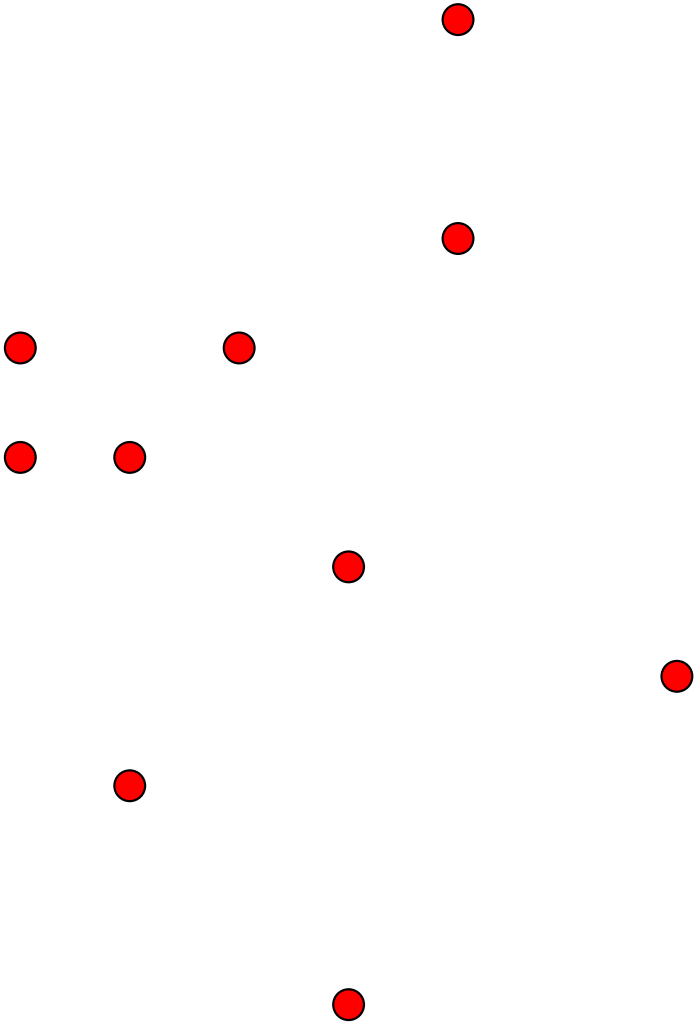
# specification of input and output

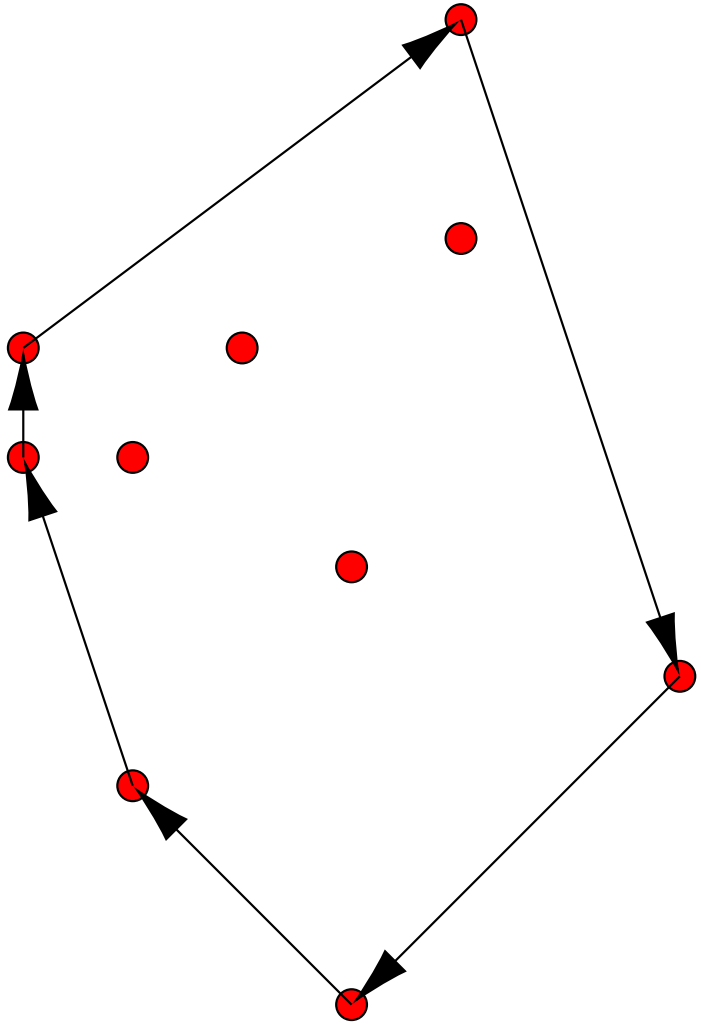
Input: $n$ points in $\mathbb{R}^2$, $S = \{p_0, p_1, \ldots, p_{n-1}\}$, $\#S = n < \infty$.
Output: list of points sorted in clockwise fashion
so two consecutive points in the list span an edge:
$L = [q_0, q_1, \ldots, q_{m-1}]$, $\left(q_i, q_{i+1 \bmod m}\right)$ spans an edge,
for $i = 0, 1, \ldots, m-1$.

## Definition (convex hull as a list)

A list $L$ of $m$ points, $L = [p_0, p_1, \ldots, p_{m-1}]$, defines a *convex hull* if for any consecutive pair of points $\left(p_i, p_{i+1 \bmod m}\right)$, all points lie at the same side of the line through $p_i$ and $p_{i+1 \bmod m}$.

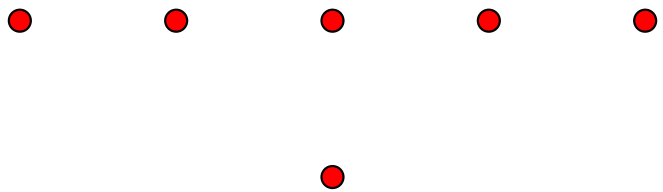
# a general case – ten random points





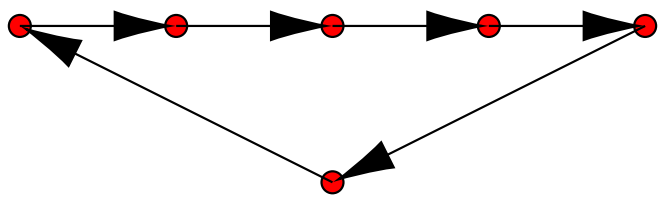
# first degenerate input: lattice polygons

Every point in a lattice polygon has integer coordinates, e.g.:

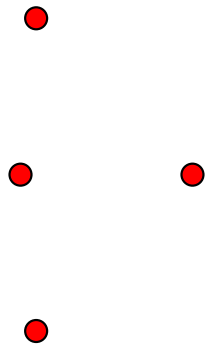


Origin: $p(x, y) = x^0y^1 + x^1y^1 + x^2y^1 + x^3y^1 + x^4y^1 + x^2y^0$.

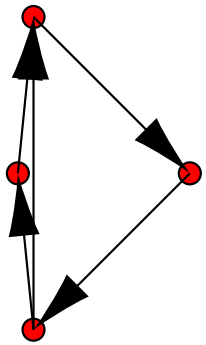
Problem: same edge is reported multiple times, as below:

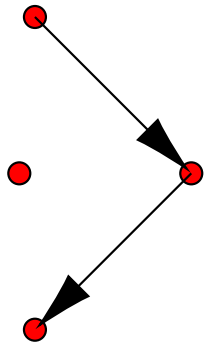


# second degenerate input: approximate coordinates



Problem: report too many or too few edges.





# general position and robustness

What counts as “degenerate” should be well defined.

## Definition (input in general position)

The input set $S$ is *in general position* if no three points in $S$ are collinear (lie on the same line).

We call an input not in general position *degenerate*.

## Definition (robust algorithm)

An algorithm is *robust* if it does not fail for small perturbations of degenerate inputs.

# Planar Convex Hull Algorithms

1. The Convex Hull
   - problem statement
   - the Graham-Andrew algorithm in CGAL
2. an Incremental Algorithm
   - the lower and upper hull
   - formulation of the algorithm
   - correctness
   - cost analysis

# the Computational Geometry Algorithms Library

*Who managed to install CGAL on their computer?*

An example of the Graham-Andrew algorithm is available in the folder
`cgal/Convex_hull_2/examples`
`ch_from_cin_to_cout.cpp`

Browse through the code at `https://github.com/CGAL/`.

The online documentation of cgal is extensive.
`https://doc.cgal.org/latest/Convex_hull_2/`
`citelist.html` lists research papers ranging from 1972 to 1996.

*Run the code!*

# Planar Convex Hull Algorithms

1. The Convex Hull
   - problem statement
   - the Graham-Andrew algorithm in CGAL

2. an Incremental Algorithm
   - the lower and upper hull
   - formulation of the algorithm
   - correctness
   - cost analysis

# idea for the algorithm

**Definition (an incremental algorithm)**

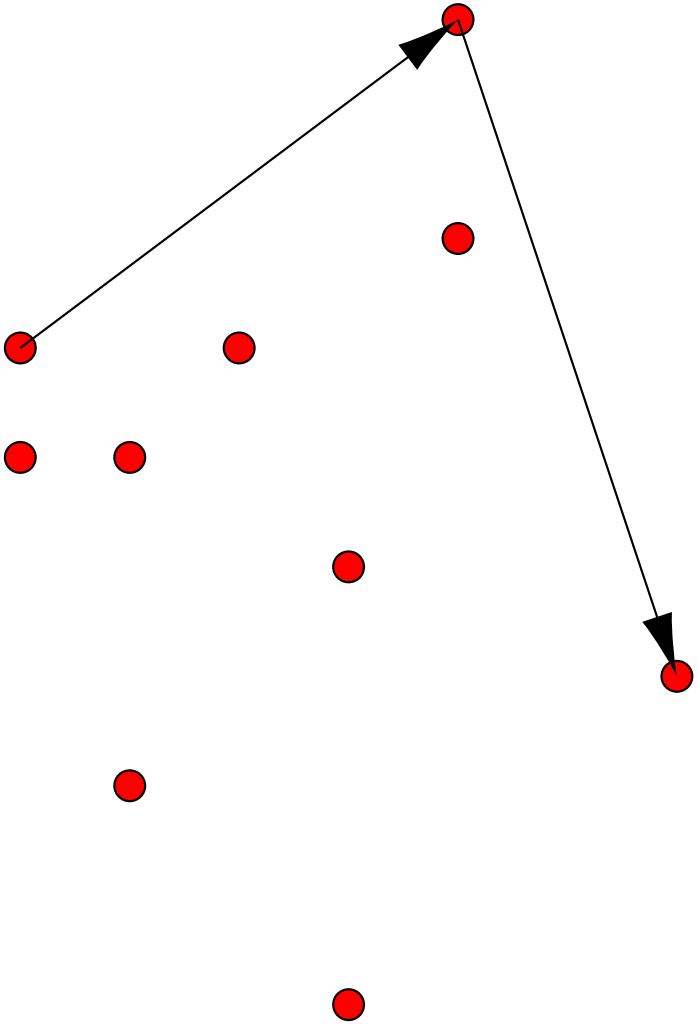
Given an input, *an incremental algorithm* updates the output successively taking more pieces of the input.

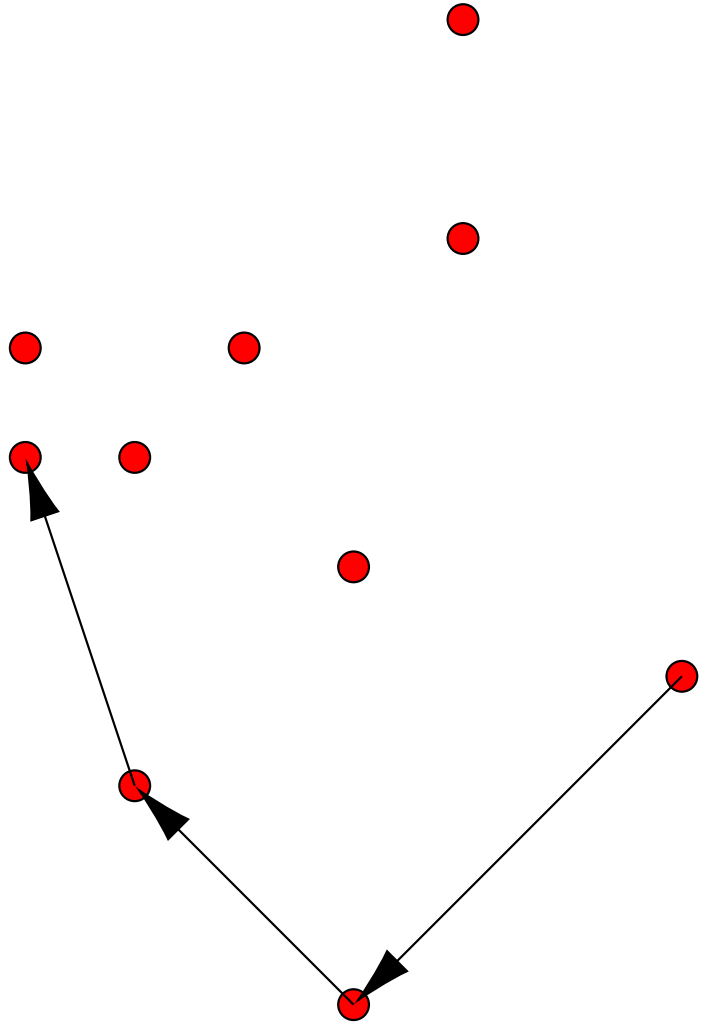
A standard algorithmic design technique:

1. first focus on the base case(s),
2. proceed by induction to the next case.

For the convex hull problem, what is (are) the base case(s)?

# upper and lower hull





# upper and lower hull – definitions

## Definition (upper hull)

Given $L$ a list of points of the convex hull ordered in clockwise fashion, *the upper hull* is the sublist of $L$ running from the leftmost to the rightmost point in $L$.

## Definition (lower hull)

Given $L$ a list of points of the convex hull ordered in clockwise fashion, *the lower hull* is the sublist of $L$ running from the rightmost to the leftmost point in $L$.

# four steps

Input: $n$ points in $\mathbb{R}^2$, $S = \{p_0, p_1, \ldots, p_{n-1}\}$, $\#S = n < \infty$.

1. Sort the point in $S$ by their $x$-coordinate.
2. Compute the upper hull, walking from left to right.
3. Compute the lower hull, walking from right to left.
4. Add lower to upper hull.

# Planar Convex Hull Algorithms

1. The Convex Hull
   - problem statement
   - the Graham-Andrew algorithm in CGAL
2. an Incremental Algorithm
   - the lower and upper hull
   - formulation of the algorithm
   - correctness
   - cost analysis

# making turns

In this incremental algorithm, we work point by point.

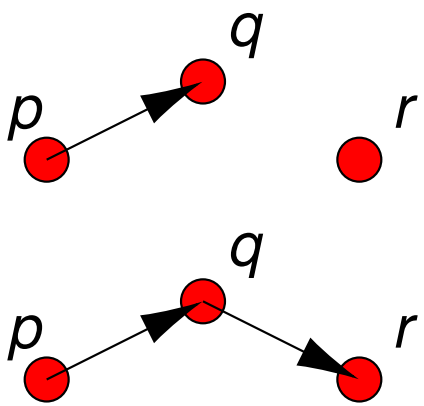
After sorting by $x$-coordinate, we have a *list* of points, so taking the next point in the list is well defined.

Consider the computation of the lower hull:

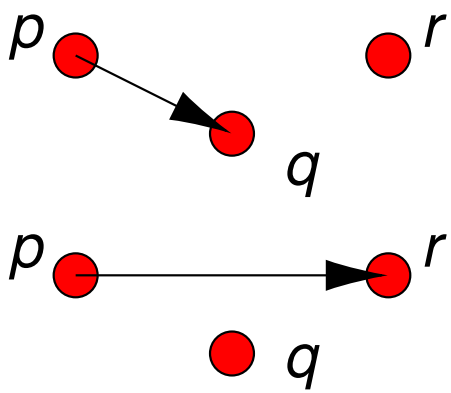
1. we have the edge $(p, q)$, $p = (p_x, p_y)$, $q = (q_x, q_y)$,
2. the next point is $r$, $r = (r_x, r_y)$, and we have: $r_x > q_x > p_x$.

Cases:

1) we make a right turn:



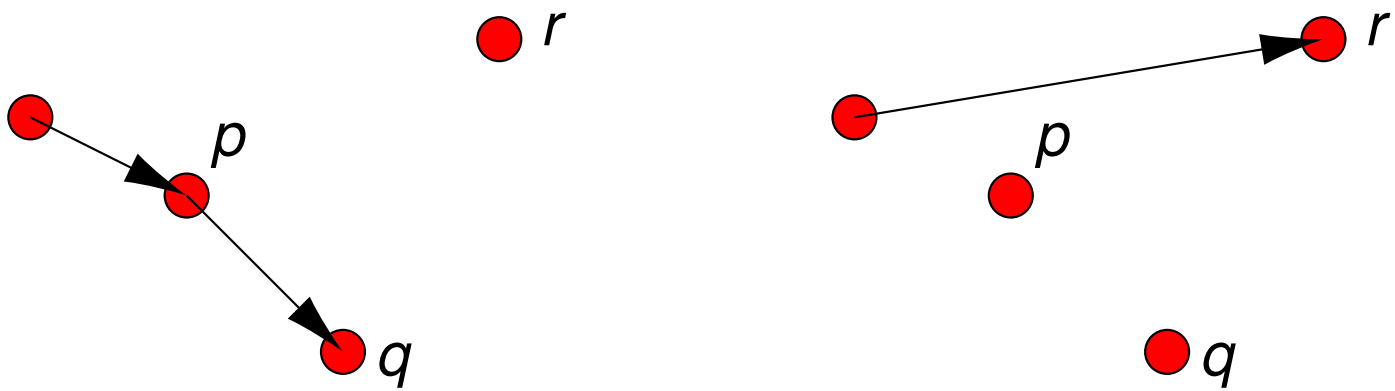

2) we make a left turn:




3) ?

# making turns – the cases continued

1. At a right turn, we have a new edge $(q, r)$.
2. At a left turn, we may have to delete more points:




   Not only $q$, but also $p$ needs to be removed.
3. We can also go straight, replacing $(p, q)$ by $(p, r)$.

# the upper hull algorithm

Input: $S = [p_0, p_1, \ldots, p_{n-1}]$, sorted on $x$-coordinate, $n > 1$.
Output: $L$ is the upper hull of $S$.

0. $L = [p_0, p_1]$
1. for $i$ from 2 to $n-1$ do
2. $\quad L = L + [p_i]$
3. $\quad$ while $\#L > 2$
4. $\quad\quad$ and last 3 points $p, q, r$ of $L$ do not make a right turn
5. $\quad$ do delete $q$ from $L$

Exercise 1: Formulate the lower hull algorithm.
Exercise 2: Modify the upper hull algorithm for collinear points.

# Planar Convex Hull Algorithms

1. The Convex Hull
   - problem statement
   - the Graham-Andrew algorithm in CGAL
2. an Incremental Algorithm
   - the lower and upper hull
   - formulation of the algorithm
   - correctness
   - cost analysis

# correctness

## Lemma (correct upper hull algorithm)

*The upper hull algorithm is correct for $S$ in general position.*

*Proof.* By induction on $n = \#S$.

1. Base case: $n = 2$ is correct, only step 0, $L = S$.
2. Assume correct for $i - 1$, we add $p_i$.
   1. Induction hypothesis: $L$ is the upper hull of $[p_0, \ldots, p_{i-1}]$.
   2. What we need to show is:

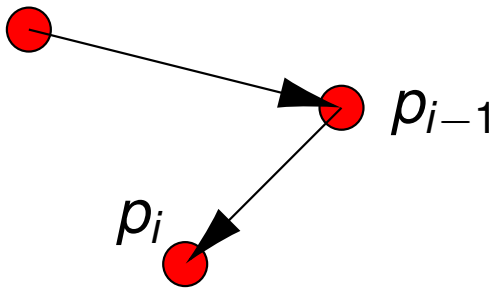
      after the while loop, $L$ is the upper hull of $[p_0, \ldots, p_i]$.

# proof of Lemma continued

3. while $\#L > 2$
4. and last 3 points $p, q, r$ of $L$ do not make a right turn
5. do delete $q$ from $L$

By contradiction, assume $L$ is not the upper hull of $[p_0, \ldots, p_i]$.
This implies: $p_i$ lies below the upper hull of $[p_0, \ldots, p_{i-1}]$.

By the while loop, all points in $L$ make a right turn.




The $x$-coordinate of $p_i$ is less than the $x$-coordinate of $p_{i-1}$.
We have a contradiction: points in $S$ are sorted on $x$-coordinate.
Our assumption, $L$ is not the upper hull of $[p_0, \ldots, p_i]$, is wrong.

Q.E.D.

# correctness for related, modified versions

Exercise 3: Show that the lower hull algorithm
is correct for points in general position.

Exercise 4: Show that the modified upper hull algorithm,
modified to handle collinear points is correct.
You may assume that all input points have exact integer coordinates.

# Planar Convex Hull Algorithms

1. The Convex Hull
   - problem statement
   - the Graham-Andrew algorithm in CGAL
2. an Incremental Algorithm
   - the lower and upper hull
   - formulation of the algorithm
   - correctness
   - cost analysis

# asymptotic upper bounds: big $O$, $O(\cdot)$

Let $T(n)$ be the worst case running time for input size $n$.

## Definition (big $O$ asymptotic upper bound)

For functions $T(n)$ and $f(n)$, $T(n)$ *is* $O(f(n))$ if there exists a constant $c > 0$, independent of $n$: $T(n) \leq c\,f(n)$, as $n \to \infty$, for all $n \geq n_0$, for a fixed constant value $n_0$.

Interpretation of the constants:

1. $n_0$: fixed cost of initialization,
2. $c$: fixed multiplier for the cost of $O(1)$ steps.

An algorithm is efficient if it runs in polynomial time.

1. $O(n)$: double the input, double the time to compute the output,
2. $O(n^2)$: double the input, multiply the time by four,
3. $O(n^3)$: double the input, multiply the time by eight.

Logarithmic or sublinear time: $O(\log(n))$.

# cost of the upper hull algorithm

0. $L = [p_0, p_1]$
1. for $i$ from 2 to $n-1$ do
2. $\quad L = L + [p_i]$
3. $\quad$ while $\#L > 2$
4. $\quad\quad$ and last 3 points $p, q, r$ of $L$ do not make a right turn
5. $\quad$ do delete $q$ from $L$

**Lemma (cost of the upper hull algorithm)**

*For $n$ points, the upper hull algorithm takes $O(n)$ time.*

*Proof.* The for loop does $n-2$ steps, but the while loop may be execute more than once and lead to $O(n^2)$.
Observe that every execution of the while deletes one point.
The while loop can never be executed more than $n$ times.

Q.E.D.

# cost of the convex hull algorithm

**Theorem (cost of the convex hull algorithm)**

*The convex hull algorithm for a set of n points runs in $O(n\log(n))$ time.*

*Proof.*

1. Sorting points on the $x$-coordinate takes $O(n\log(n))$ time.
2. The upper hull algorithm runs in $O(n)$ time.
3. The lower hull algorithm runs in $O(n)$ time.
4. Adding upper and lower hulls runs in $O(n)$ time.

$O(n\log(n)) + O(n) + O(n) + O(n)$ is $O(n\log(n))$.

Q.E.D.

# exercises

We covered the first chapter in the textbook.

Consider the following activities, listed below.

1. Install CGAL on your computer (if not already done).
   Run the examples in the `/Convex_hull_2/examples` folder of CGAL and browse the documentation.
2. Write the solutions to Exercises 1 through 4.
3. Consider the exercises 4,7,9 in the textbook.